



TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

TRABAJO INTEGRADOR:

Gestión de Datos de Países en Python:
filtros, ordenamientos y
estadísticas

Comisión 12

Joaquín Villalba - joaquinwillalba8@gmail.com

1/11/2025

PROGRAMACIÓN I

1. Marco teórico

Listas: son estructuras de datos que permiten almacenar varios elementos en un solo objeto, manteniendo un orden y siendo modificables. []

Ejemplo:

```
países = [ {"nombre": "Argentina", "poblacion": 45376763, "superficie":  
2780400, "continente": "América"} ]
```

Diccionarios: son estructuras de datos no ordenadas que almacena información en pares clave-valor. {}

Ejemplo:

```
{ "nombre": "Argentina", "poblacion": 45376763, "superficie": 2780400, "continente":  
"América" }
```

Funciones: son bloques de códigos reutilizables que cumplen tareas específicas

Ejemplo:

```
agregar_pais()
```

Condicionales: en Python son como las bifurcaciones en un camino: te permiten tomar decisiones según si una condición es verdadera o falsa

Ejemplo:

```
if opcion == "1":  
    agregar_pais(países)
```

Ordenamientos: organizan los datos de una lista según un criterio determinado.

Ejemplo:

```
ordenados = sorted(paises, key=lambda x: x[clave], reverse=desc)
```

Estadísticas básicas: permiten obtener información general sobre un conjunto de datos, como valores máximos, mínimos o promedios.

Ejemplo:

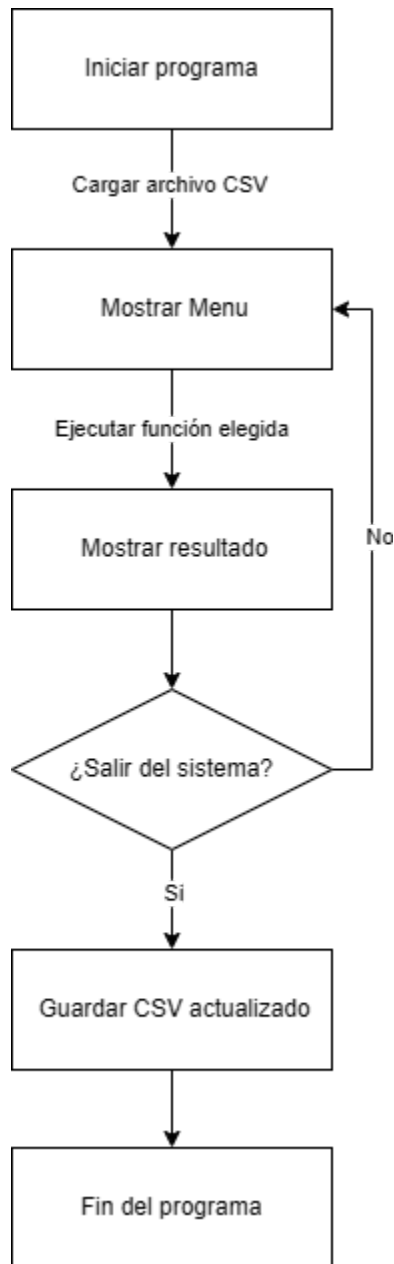
```
mayor = max(paises, key=lambda x: x["poblacion"])
```

Archivos CSV: Es un formato de texto que almacena datos tabulares separados por comas. Se utilizan para guardar, leer, modificar, importar y exportar información.

Ejemplo:

```
def guardar_paises(paises):  
    with open(ARCHIVO, "w", newline="", encoding="utf-8") as f:  
        campos = ["nombre", "poblacion", "superficie", "continente"]  
        escritor = csv.DictWriter(f, fieldnames=campos)  
        escritor.writeheader()  
        escritor.writerows(paises)
```

Flujo de operaciones principales



2. Objetivo del trabajo

El propósito principal de este trabajo práctico integrador es crear un programa en Python que permitirá la gestión de datos sobre países, empleando estructuras como listas y diccionarios, y aplicando funciones, condicionales, bucles y manipulación de archivos CSV.

El objetivo es demostrar la capacidad para diseñar, implementar y documentar una aplicación completa en Python, consolidando así los conocimientos adquiridos en Programación 1.

3. Diseño del caso práctico

Para el desarrollo del sistema de gestión de países, se diseñó una estructura modular que permite administrar la información de manera clara. El diseño se centró en la reutilización del código y el uso de estructuras de datos adecuadas para almacenar y procesar la información.

El programa está compuesto por tres partes principales:

1. **Funciones de archivos:** lectura y escritura del archivo `paises.csv` mediante el módulo `csv`, garantizando la persistencia de la información.
2. **Funciones del sistema:** contienen toda la lógica del programa: agregar, actualizar, buscar, filtrar, ordenar y generar estadísticas. Cada función tiene una responsabilidad específica.
3. **Interfaz por consola (Menú principal):** permite al usuario interactuar con las diferentes funciones del sistema de forma intuitiva.

Los países se representan en una lista de diccionarios, donde cada elemento contiene las claves `nombre`, `poblacion`, `superficie` y `continente`:

Ejemplo:

```
{  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}
```

4. Resultados Obtenidos

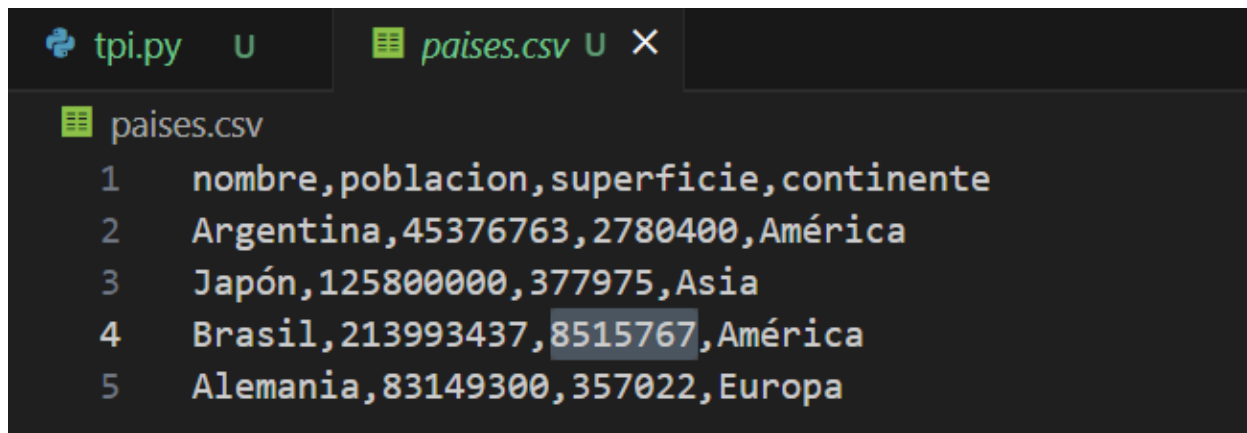
Todas las funciones principales, como agregar, actualizar, buscar, filtrar, ordenar y generar estadísticas, se ejecutaron correctamente y mostraron los resultados esperados. La persistencia mediante archivos CSV funcionó sin inconvenientes, garantizando que los datos se mantuvieran actualizados entre sesiones.

En una etapa temprana del desarrollo consideré implementar el programa usando clases en vez de diccionarios, pero decidí usar estos últimos por ser más simples y directos para la manipulación de datos en este caso.

Una de las dudas que tuve es si debería actualizar automáticamente el csv al agregar o actualizar un país, como en el segundo parcial, pero como no había una aclaración explícita decidí que el dato modificado se mantiene en memoria, hasta que se elige la opción de guardar y salir del sistema. Otra decisión que tomé por el mismo motivo es hacer validaciones sobre nombres con comparación insensible a mayúsculas/minúsculas y espacios redundantes, pero no sobre acentos.

Este trabajo lo realice completamente solo ya que si bien tenía un compañero este dejó de responder mis mensajes por lo cual entiendo que me abandonó, de forma irresponsable. Al ser un programador de varios años de experiencia decidí continuar y presentar el trabajo de forma individual

Capturas del programa en funcionamiento:



```
tpi.py  U  países.csv  U  X
países.csv
1  nombre,poblacion,superficie,continente
2  Argentina,45376763,2780400,América
3  Japón,125800000,377975,Asia
4  Brasil,213993437,8515767,América
5  Alemania,83149300,357022,Europa
```

```
*****
```

```
===== GESTIÓN DE PAISES =====
```

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
7. Ordenar países
8. Mostrar estadísticas
9. Guardar y salir

```
*****
```

```
Ingrese opcion: 1  
Nombre del país: Chile  
Población: 19000000  
Superficie (km²): 756626  
Continente: América  
País agregado con éxito.
```

```
Ingrese opcion: 2  
Ingrese el nombre del país a actualizar: chile  
Actualizando datos de: Chile  
Nueva población: 19001000  
Nueva superficie (km²):  
Error: Debe ingresar un número válido para la superficie. Intente nuevamente.  
Nueva superficie (km²): 756626  
Datos actualizados con éxito.
```

```
Ingrese opcion: 3
Buscar país (nombre o parte): chile

Se encontraron 1 resultado(s):
{'nombre': 'Chile', 'poblacion': 19001000, 'superficie': 756626, 'continente': 'América'}
```

```
Ingrese opcion: 4
Ingrese continente: América

Países encontrados en 'América':
{'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400, 'continente': 'América'}
{'nombre': 'Brasil', 'poblacion': 213993437, 'superficie': 8515767, 'continente': 'América'}
{'nombre': 'Chile', 'poblacion': 19001000, 'superficie': 756626, 'continente': 'América'}
```

```
Ingrese opcion: 5
Ingrese poblacion mínimo: 20001000
Ingrese poblacion máximo: 200000000

Países con poblacion entre 20001000 y 200000000:
{'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400, 'continente': 'América'}
{'nombre': 'Japón', 'poblacion': 125800000, 'superficie': 377975, 'continente': 'Asia'}
{'nombre': 'Alemania', 'poblacion': 83149300, 'superficie': 357022, 'continente': 'Europa'}
```

```
Ingrese opcion: 6
Ingrese superficie mínimo: 8515767
Ingrese superficie máximo: 8515768

Países con superficie entre 8515767 y 8515768:
{'nombre': 'Brasil', 'poblacion': 213993437, 'superficie': 8515767, 'continente': 'América'}
```



```
Ingrese opcion: 7
1. Nombre 2. Población 3. Superficie
Ordenar por: 3
¿Descendente? (s/n): s
{'nombre': 'Brasil', 'poblacion': 213993437, 'superficie': 8515767, 'continente': 'América'}
{'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400, 'continente': 'América'}
{'nombre': 'Chile', 'poblacion': 19001000, 'superficie': 756626, 'continente': 'América'}
{'nombre': 'Japón', 'poblacion': 125800000, 'superficie': 377975, 'continente': 'Asia'}
{'nombre': 'Alemania', 'poblacion': 83149300, 'superficie': 357022, 'continente': 'Europa' }
```

```
Ingrese opcion: 8
País con mayor población: Brasil (213993437)
País con menor población: Chile (19001000)
Promedio de población: 97464100.00
Promedio de superficie: 2557558.00
Cantidad de países por continente:
- América: 3
- Asia: 1
- Europa : 1
```

```
tpi.py  U  pais.es.csv  U  X

pais.es.csv
1  nombre,poblacion,superficie,continente
2  Argentina,45376763,2780400,América
3  Japón,125800000,377975,Asia
4  Brasil,213993437,8515767,América
5  Alemania,83149300,357022,Europa |
6  Chile,19001000,756626,América
7

PROBLEMS  OUTPUT  DEBUG CONSOLE  PORTS  TERMINAL

Promedio de población: 97464100.00
Promedio de superficie: 2557558.00
Cantidad de países por continente:
- América: 3
- Asia: 1
- Europa : 1

*****

===== GESTIÓN DE PAISES =====
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
7. Ordenar países
8. Mostrar estadísticas
9. Guardar y salir

*****

Ingrese opcion: 9
Datos guardados. Saliendo...
```

5. Conclusiones

Aunque tengo experiencia profesional como programador, este trabajo me permitió practicar el desarrollo de programas interactivos por consola y trabajar con archivos CSV, herramientas que no utilizo habitualmente en mi entorno laboral diario.

Estas herramientas resultan importantes porque permiten estructurar datos de manera clara, persistente y flexible, lo que es fundamental al manejar información que debe persistir entre sesiones. Por ejemplo, el uso de listas y diccionarios facilitó filtrar, ordenar y buscar datos de países, mientras que los archivos CSV aseguraron la persistencia de los datos de manera simple y compatible con otras aplicaciones.

El proyecto demuestra cómo operaciones básicas de lectura/escritura y manipulación de estructuras de datos permiten desarrollar un sistema funcional, modular y mantenible.

En conclusión, este trabajo me permitió aplicar conocimientos previos en un contexto distinto, adquirir nuevas habilidades prácticas en consola y archivos CSV, y comprender la importancia de planificar, modularizar y documentar correctamente un programa para asegurar su correcta ejecución y mantenimiento.

6. Bibliografía

- Python Software Foundation. (2025). *Python Documentation*. Recuperado de <https://docs.python.org/3/>
- Universidad Tecnológica Nacional (UTN). (2025). Material del Campus Virtual — Programación I. Contenidos teóricos.

7. Repositorio

<https://github.com/jnvillalba/UTN-TUPaD-Programacion1/tree/feature/TPI/TPI>